Building, Not Just Doing: The Accumulator

Class 9 — In-Class Work-Along Sheet (Part 1 — the accumulator)

An **accumulator** is a variable that lives *outside* the loop and **builds up a result** across passes —

a sum, a count, a product, a reversed number. You met one already in last class's factorial problem;

today you name it and master it.

How to use this sheet: answer every "predict" box **before** you run the code. The ⟳ symbol marks a question that reaches back to an earlier class.

Part 1 — Meeting the Accumulator

Q1 — A running total

Predict first, then run:

```
total = 0
i = 1
while i <= 5:
    total += i
    i += 1
print(total)
```

```
Predicted: _____________     Actual: _____________
```

(a) Trace the value of `total` right after each pass:

```
after pass:  1 → ____   2 → ____   3 → ____   4 → ____   5 → ____
```

(b) `total += i` is shorthand for `total = _____________`.

Q2 — Where the accumulator lives (the #1 trap)

Same loop, but one line moved. **Predict the output**, then run:

```
i = 1
while i <= 5:
    total = 0
    total += i
    i += 1
print(total)
```

```
Predicted: _____________     Actual: _____________
```

(a) Why does this print `5` and not `15`? What does `total = 0` do on every single pass?

```
_____________________________________________________________________
```

(b) State the rule in your own words — *where* must an accumulator be initialised?

```
_____________________________________________________________________
```

Q3 — Counting is accumulating

Fill the blanks so this counts how many numbers from 1 to 10 are **even** (answer should be 5):

```
count = ____
i = 1
while i <= 10:
    if i % 2 == ____:
        count ____ 1
    i += 1
print(count)
```

(a) A counter is just an accumulator that adds _____ each time — but only when a _____ is true.

Q4 — Why does a product start at 1?

Predict what each prints:

Version A

```
product = 0
i = 1
while i <= 5:
    product *= i
    i += 1
print(product)
```

Predicted: `_____` Actual: `_____`

Version B

```
product = 1
i = 1
while i <= 5:
    product *= i
    i += 1
print(product)
```

Predicted: `_____` Actual: `_____`

(a) Version A is stuck. In one sentence, why can a product accumulator never escape if it starts at 0?

```
_____________________________________________________________________
```

(b) Complete the table of starting values:

```
operation   | start the accumulator at
------------+--------------------------
sum         |  ______
product     |  ______
count       |  ______
```

Part 1 (cont.) — Taking a number apart ⟳

Q5 — ⟳ from Class 8 (`%` and `//`), now in a loop

First recall (no running): for `n = 4827`: `n % 10` = _____ `n // 10` = _____

Now trace this digit-sum loop **by hand** — fill one row per pass:

```
n = 4827
digit_sum = 0
while n > 0:
    digit_sum += n % 10
    n //= 10
print(digit_sum)
```

```
   n      | n % 10 | digit_sum | n // 10
   -------+--------+-----------+---------
   4827   |   7    |    7      |   482
   482    | _______ |  _______  |  _______
   48     | _______ |  _______  |  _______
   4      | _______ |  _______  |  _______
```

(a) Final value printed: _____

(b) Which variable is the accumulator here, and which two operators do the "take a digit / drop a digit" work?

```
accumulator: _____________     operators: _____________
```

Q6 — Reverse the digits

Predict the output, then run:

```
n = 4827
rev = 0
while n > 0:
    rev = rev * 10 + n % 10
    n //= 10
print(rev)
```

```
Predicted: _____________     Actual: _____________
```

(a) Fill the build-up of `rev`: 0 → _____ → _____ → _____ → _____

(b) ⟳ Class 8 — the condition is `while n > 0`. What goes wrong if you write `while n >= 0` instead?

```
_____________________________________________________________________
```

Q7 — Generate before you're told

You have **not** been shown a "count the digits" loop. Build one, using the same `%` / `//` skeleton.

Write a loop that reads an integer `n` and prints **how many digits** it has (e.g. `4827 → 4`):

```
_____________________________________________
_____________________________________________
_____________________________________________
_____________________________________________
```

(a) What is your accumulator, and what do you start it at? _____ **(b)** What line shrinks `n` toward 0 each pass? _____

Spaced recall — reach back ⟳

Q8 — ⟳ Class 8 (the flag) vs today (the accumulator)

Last class you invented a **flag** (a True/False memory). Today you used an **accumulator** (a building-up number). Both are *techniques*, not Python features.

(a) In one line each, what is the difference in what they *remember*?

```
flag remembers: _____     accumulator remembers: _____
```

(b) Neither `flag` nor `accumulator` is a Python keyword. So where do these ideas live, and why can you use them in any language?

```
_____
```

Q9 — Books closed — free recall

Close the sheet. From memory:

(a) What is an accumulator?

```
_____
```

(b) Starting value for a **sum**: _____ for a **product**: _____ — and why are they different?

```
_____
```

(c) `n % 10` gives _____ ; `n // 10` gives _____.

Part 2 — GCD & LCM by Loops

You found GCD and LCM by hand in school. Now make a **program** do it — with nothing but a loop and `%`.

Every solution here is the Part 1 accumulator in disguise — a running best — plus `break` from Class 8.

Q10 — GCD by brute force

Predict first, then run:

```
a = 12
b = 18
gcd = 1
i = 1
while i <= min(a, b):
    if a % i == 0 and b % i == 0:
        gcd = i
    i += 1
print(gcd)
```

```
Predicted: _____     Actual: _____
```

(a) List every value `i` takes that *updates* `gcd` (i.e. divides both 12 and 18): _____

(b) The loop runs up to `min(a, b)`. Why can a common divisor never be larger than the smaller number?

```
_____
```

(c) What would this print for `a = 7, b = 5`? _____ Why?

Q11 — Make it stop early

The loop above keeps going even after it has found the answer. Fill the blanks so it searches **downward** and stops at the first common divisor (which is automatically the largest):

```
a = 12
b = 18
i = _____________          # start here
while i >= 1:
    if a % i == 0 and b % i == 0:
        print(i)
        _____________      # leave immediately
    i ____ 1               # move toward 1
```

(a) Going downward, why is the first common divisor you hit guaranteed to be the greatest?

```
_____________________________________________________________________
```

Q12 — LCM by brute force

Fill the blanks so this finds the **least** common multiple of `a` and `b`:

```
a = 4
b = 6
c = _____________          # smallest value LCM could possibly be
while c <= _____________:  # largest value it could possibly be
    if c % a == 0 and c % b == 0:
        print(c)
        break
    c += 1
```

(a) Predicted output: _____

(b) Why is `a * b` always a safe upper bound — i.e. why is it guaranteed to be a common multiple?

```
_____________________________________________________________________
```

(c) For a GCD you searched high→low; for an LCM you search low→high. In one line, why the opposite directions?

```
_____________________________________________________________________
```

Q13 — The identity `GCD × LCM = a × b`

(a) Verify it numerically for `a = 12, b = 18` (you found `gcd = 6` and `lcm = 36`):

```
gcd × lcm = ____ × ____ = ____      a × b = 12 × 18 = ____      equal? ____
```

(b) Fill in the proof, instance `a = 12, b = 18`, `g = 6`:

```
12 = 6 × ____      18 = 6 × ____      (the two blanks share no common factor)
lcm = 6 × ____ × ____ = ______
```

(c) The identity gives a shortcut for LCM with no second search. Complete it:

```
lcm = a * b ______ gcd      # which operator keeps it an exact integer?
```

Q14 — Euclid's method: trace and compare

Trace this by hand (you saw it in school as the division method):

```
a = 48
b = 18
while b != 0:
    r = a % b
    a = b
    b = r
print(a)
```

```
   a    | b    | r = a % b
   -----+------+----------
   48   | 18   |  ______
   18   | ____ |  ______
   ____ | ____ |  ______
   ____ |  0   |  (stop)
```

(a) Final value printed (the GCD): _____

(b) Euclid took _____ steps here. Brute force (Q10) would have looped up to `min(48,18)` = _____ times. Which idea names this difference in "amount of work"? _____

Q15 — Bridge problem: GCD of a whole list

Now the real challenge: given `nums = [12, 18, 24]`, find the GCD of all of them.

(a) The idea is a running GCD (the accumulator again): start with the first number, then fold in each next one. Fill the blanks:

```
gcd_so_far = nums[0]
combine gcd_so_far with nums[1]  ->  gcd( ____ , ____ ) = ______
combine that with nums[2]        ->  gcd( ____ , 24 )   = ______
final answer = ______
```

(b) To compute each `gcd(...)` above you need a loop — running *inside* the loop that walks the list. What is that structure called? _____ *(it's the very next topic)*

(c) ⟳ Spotting an incomplete spec (think like an interviewer): before coding "GCD of a list", name two questions you'd ask about the input.

```
Q(i) : _____________________________________________________
Q(ii): _____________________________________________________
```

Spaced recall — reach back ⟳

Q16 — ⟳ the accumulator, everywhere

(a) Q10's `gcd` variable and Part 1's `total` variable are the same technique wearing different clothes. What is that technique, and what does each one "build up"?

```
technique: _____________   |   total builds: _____________   |   gcd builds: _____________
```

(b) ⟳ Class 8 — Q11 and Q12 both used `break`. In one line, what does `break` buy you in these searches?

```
_____________________________________________________________
```

Q17 — Books closed — free recall

Close the sheet. From memory:

(a) To brute-force a GCD: loop `i` from 1 to _____ , keep `i` whenever _____.

(b) The identity linking GCD, LCM and the two numbers: _____ , so `lcm = _____`.

(c) Brute force vs Euclid on big numbers — which does far less work, and what is that idea called?

_____

Part 3 — Nested Loops & Flow of Control

A **loop inside a loop**. The rule that unlocks all of them: for every single pass of the outer loop, the inner loop runs **all the way to completion**, re-initialising each time. Predict every output before running.

Q18 — Draw the flowchart

For the loop below, **draw the flowchart** in the box: two decision diamonds (`i <= n ?` and `j <= i ?`), the True/False branches, and the two return-arrows (inner back to its own test, outer back to its own test).

```
i = 1
while i <= n:
    j = 1
    while j <= i:
        print(j, end=" ")
        j += 1
    print()
    i += 1
```

```
 ____________________________________________________________________
|                                                                    |
|                                                                    |
|                                                                    |
|                                                                    |
|                    ( draw your flowchart here )                    |
|                                                                    |
|                                                                    |
|                                                                    |
|____________________________________________________________________|
```

(a) Where in your flowchart does `j = 1` sit — inside or outside the outer loop's body? _____

Q19 — Predict: easy

```
i = 1
while i <= 2:
    j = 1
    while j <= 3:
        print(i * j, end=" ")
        j += 1
    print()
    i += 1
```

```
Predicted:                Actual:
_____________             _____________
_____________             _____________
```

Q20 — Predict: medium

```
i = 1
while i <= 4:
    j = 1
    while j <= i:
        print(i, end=" ")
        j += 1
    print()
    i += 1
```

```
Predicted:          Actual:
_____________       _____________
_____________       _____________
_____________       _____________
_____________       _____________
```

(a) In one line: why does row `i` contain the number `i` repeated `i` times?

```
_____________________________________________________________________
```

Q21 — Predict: hard (track counter, arithmetic, and branch)

```
i = 1
while i <= 4:
    j = 1
    while j <= 4:
        if (i * j) % 2 == 0:
            print("E", end=" ")
        else:
            print("O", end=" ")
        j += 1
    print()
    i += 1
```

```
Predicted:              Actual:
_______________         _______________
_______________         _______________
_______________         _______________
_______________         _______________
```

Q22 — Predict: `break` inside the nest

```
i = 1
while i <= 3:
    j = 1
    while j <= 4:
        if i * j >= 4:
            break
        print(i, j)
        j += 1
    i += 1
```

Predicted output (one `i j` pair per line):

```
_____________________________________________________________________
_____________________________________________________________________
```

(a) On which outer passes did the inner loop `break` *before* printing anything? _____

Q23 — Explain it in your own words

A `break` is placed inside the **inner** loop of a nested pair. In 2–3 sentences, explain exactly which loop stops and what the outer loop does next. Then state what you'd need to exit **both** loops at once.

```
_____________________________________________________________
_____________________________________________________________
_____________________________________________________________
```

Q24 — Predict: a nested loop, then an independent loop

```
i = 1
while i <= 2:
    a = 1
    while a <= i:
        b = 1
        while b <= 2:
            print("*", end="")
            b += 1
        a += 1
    print("|", end="")
    c = 1
    while c <= 3:
        print("-", end="")
        c += 1
    print()
    i += 1
```

```
Predicted:                Actual:
__________________        __________________
__________________        __________________
```

(a) Inside one outer pass, name the two structures that run, in order: _____ then _____

Q25 — Predict: two sequential inner blocks with branches

```
i = 1
while i <= 3:
    j = 1
    while j <= 3:
        if j <= i:
            print("#", end="")
        else:
            print(".", end="")
        j += 1
    k = 1
    while k <= 3:
        if k > i:
            print("#", end="")
        else:
            print(".", end="")
        k += 1
    print()
    i += 1
```

```
Predicted:                Actual:
__________________        __________________
__________________        __________________
__________________        __________________
```

Q26 — Generate: write the triangle yourself

Using a nested `while` loop, print this for `n = 5` (each number separated by a space):

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

```
n = 5
_____________________________________
_____________________________________
_____________________________________
_____________________________________
_____________________________________
_____________________________________
_____________________________________
```

(a) What is your inner loop's upper limit, written in terms of the outer counter? _____

Q27 — Books closed — free recall (Part 3)

Close the sheet. From memory:

(a) For one pass of the outer loop, how much of the inner loop runs? _____

(b) A `break` in the inner loop of a nested pair stops which loop(s)? _____

(c) Where must the inner loop's counter be initialised, and why?

```
_____________________________________________________________
```

The infinite-loop trap — make the control variable change

> ⟳ From Class 8: a `while` loop runs as long as its condition is True. If nothing in the body ever changes the variable that condition depends on, the loop never ends. In a deep nest this is dangerously easy to miss, and the symptom is a frozen program.

For each below, **predict whether it ever stops**. Then run it — but keep `Ctrl+C` ready to kill it.

Q28 — Two levels

```
i = 1
while i <= 3:
    j = 1
    while j <= 3:
        print(i, j)
    i += 1
```

(a) Does this program terminate? _____

(b) Which loop runs forever, and which variable never changes? _____

(c) What single line, added where, fixes it? _____

Q29 — Three levels (the kind that really bites)

```
i = 1
while i <= 2:
    j = 1
    while j <= 2:
        k = 1
        while k <= 2:
            print(i, j, k)
            k += 1
    i += 1
```

(a) The innermost (`k`) loop does update its counter — yet the program still hangs. In one or two lines, explain why.

_____
_____

(b) Which variable's missing update is the real culprit? _____

(c) Write the one-line fix, and say exactly where it goes: _____

The while-loop rule — never forget it: every `while` loop must, somewhere in its body, change the variable its condition depends on. In a nest, *every* level needs its own update — miss one and that loop spins forever (Time Limit Exceeded on a judge; a frozen terminal locally, escaped with `Ctrl+C`).